RISHIHOOD UNIVERSITY

Newton School of Technology

B.Tech. in Computer Science and Engineering

Academic Session 2026

HiggsLens

Machine Learning-Based Higgs Boson Signal Detection and Classification

Project Domain: Machine Learning, Deep Learning, Data Science and High-Energy Physics

Project Team

Student Name	Enrollment No.	Project Role
Saumya Mishra	2401020057	Documentation, Testing and Reporting
Aditya Rana	2401010034	Data Processing and Classical Machine Learning
Rachit Gupta	2401010363	Neural Network Development
Ronit Singh	2401010393	Dashboard and Visualization
Krishna	2401010237	Project Integration and Technical Decisions

Table of Contents

1. Abstract..... 3
2. Introduction..... 4
3. Background: Higgs Boson and Particle Physics..... 5
4. Problem Statement..... 7
5. Problem Explanation and Motivation..... 8
6. Project Objectives..... 9
7. Scope of the Project..... 10
8. Dataset Description and Data Card..... 11
9. Exploratory Data Analysis..... 13
10. Proposed and Implemented Methodology..... 14
11. System and Project Architecture..... 16
12. Data Preprocessing and Feature Engineering..... 17
13. Classical Machine Learning Models..... 18
14. Neural Network Model..... 19
15. Model Evaluation and Performance Metrics..... 21
16. Software Architecture and Dashboard..... 22
17. Work Division and Team Contributions..... 23
18. Implementation and Reproducibility..... 24
19. Testing and Validation..... 25
20. Results and Comparative Analysis..... 27
21. Limitations, Risks and Open Issues..... 30
22. Ethical Considerations and Scientific Interpretation..... 32
23. Future Scope..... 33
24. Conclusion..... 34
25. References..... 35
26. Appendices..... 36

Figures and tables are listed in Appendix F.

1. Abstract

Identifying rare signal events among a very large number of background events is a central problem in experimental high-energy physics. The Higgs boson, observed at the Large Hadron Collider in 2012 [1]–[3], was found within such a stream of collisions. HiggsLens studies the machine-learning side of this problem using the public HIGGS benchmark of simulated collision events [4], [5].

The project validates the raw data, draws seeded samples, creates a stratified 70/15/15 split with seed 42, and trains three model families: Logistic Regression, XGBoost and a compact multilayer perceptron (MLP) with batch normalization and dropout. Each family is evaluated on three feature groups: 21 low-level kinematic features, 7 high-level invariant-mass features and all 28 features. Results are saved in a shared artifact format, and a Streamlit dashboard, HiggsLens, displays saved results without training any model.

On the 250,000-event subset, the MLP trained on all 28 features reaches a test ROC-AUC of 0.9406, XGBoost 0.8924 and Logistic Regression 0.8369. On the 1,000,000-event benchmark the best model is a wider MLP with a test ROC-AUC of 0.8399, followed by XGBoost at 0.8271 and Logistic Regression at 0.6834. **The two protocols do not agree, and every model scores lower on the larger sample.** This report therefore does not declare a headline result. The gap is recorded as an open issue (Section 20.4) and must be resolved before the results are presented as final.

The study is a benchmark of simulated event classification. **It does not claim the discovery of any particle, does not reproduce the 2012 measurement, and does not produce physical probabilities.** The value of the work lies in a documented comparison of methods, a transparent data pipeline, and an honest account of what is verified and what is not.

2. Introduction

Particle physics asks how fundamental particles acquire mass and how they interact. In the Standard Model, a Higgs field fills all of space, and its excitation, the Higgs boson, was predicted in 1964 by several groups [6]. The ATLAS and CMS experiments reported a particle consistent with it at about 125 GeV in July 2012 [2], [3].

At the LHC, protons collide at very high energy and each collision yields a spray of particles. Most collisions are ordinary processes, and the rare processes of interest are hidden among them. Physicists therefore classify events as signal-like or background-like, and machine learning is now a standard tool for this task.

The HIGGS dataset, released with a study by Baldi, Sadowski and Whiteson [4], [5], is a public benchmark of simulated events for this classification problem. It makes controlled, reproducible comparison of methods possible. This report documents the HiggsLens project: the problem, the data, the methods, the implementation, the testing, the results that are verified, and the limitations that remain.

The report separates three kinds of statement throughout: what has been implemented and tested in the repository, what has been measured in saved runs, and what is still pending. Where a number could not be checked against a saved file, it is marked as pending.

3. Background: Higgs Boson and Particle Physics

3.1 The Standard Model and the Higgs field

The Standard Model describes the known elementary particles and three of the four fundamental forces. The W and Z bosons carry the weak force. For the electroweak theory to remain consistent, these carriers must be massless in the basic equations, yet they are measured to be heavy. The Higgs mechanism resolves this: a field that is non-zero everywhere in the lowest-energy state gives mass to the W and Z bosons and, through separate couplings, to the charged fermions [6].

3.2 The Higgs boson

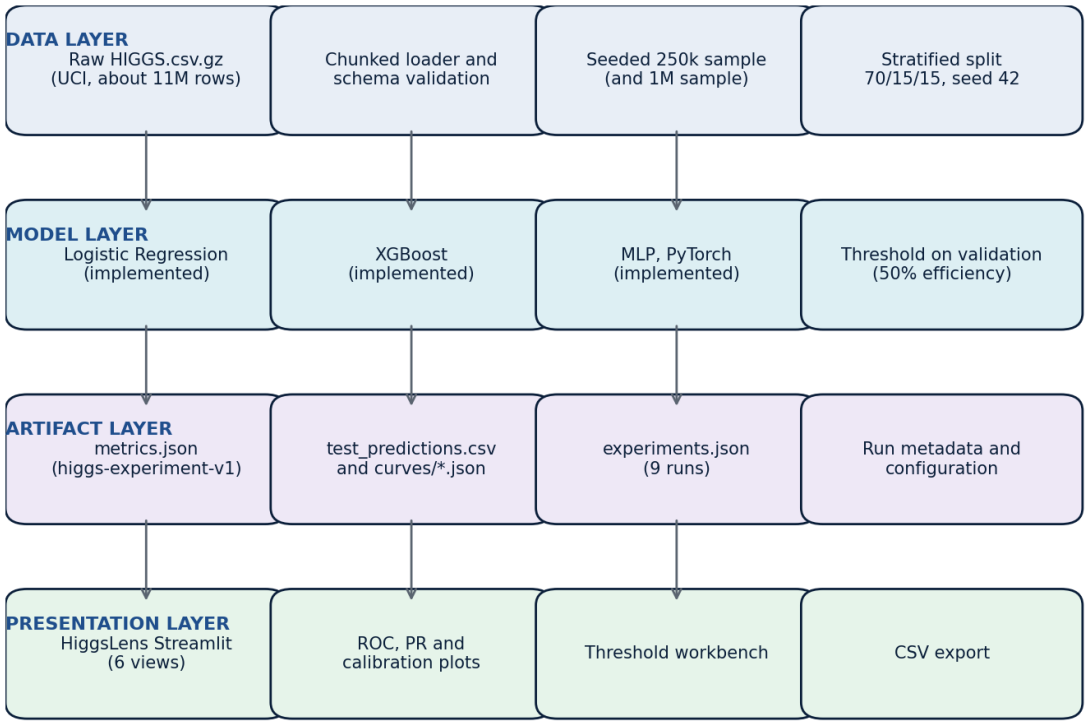
The Higgs boson is the quantum excitation of this field. It is electrically neutral and has spin zero, and its mass is about 125 GeV [2], [3]. It decays almost immediately, so it is inferred from its decay products rather than seen directly. The 2013 Nobel Prize in Physics was awarded to Englert and Higgs for the theory [7].

3.3 Why classification is needed

The Higgs boson is produced rarely compared with the total number of collisions, and background processes can leave similar patterns in the detector. Physicists must choose selections that enhance signal relative to background. This is a classification problem, which is why the HIGGS benchmark is framed as signal versus background.

3.4 What the HIGGS benchmark represents

According to the original study [4], the signal in the benchmark involves hypothetical heavy Higgs-like particles, and the background is a top-quark process that produces similar final-state particles with different kinematics. The benchmark is therefore a designed classification problem, **not a re-creation of the 2012 Higgs discovery**. All events are computer-simulated.



Green is the presentation layer. All components shown are implemented in the repository.

Figure 1. Architecture of HiggsLens as implemented. All components shown are present in the repository. The dashboard reads saved artifacts and does not train models.

4. Problem Statement

Develop and evaluate a reproducible machine-learning pipeline that classifies events in the HIGGS benchmark as signal-like or background-like from their recorded features, compares three model families across three feature groups, and presents the saved results through an analytical dashboard. Each part of this problem has to be solved for the comparison to mean anything:

- The raw file is about 11 million rows and must be read and validated without loading it fully into memory.
- A subset must be drawn in a documented, reproducible way.
- Every model must be trained, selected and tested on the same split, so that comparisons are fair.
- Performance must be measured with several metrics, because accuracy alone hides the trade-off between false positives and false negatives.
- Results must be saved in one format, so that the dashboard can display them without training anything.

5. Problem Explanation and Motivation

Each event is described by 21 low-level measurements: the transverse momentum, pseudorapidity and azimuthal angle of one charged lepton and of up to four jets; a b-tag value for each jet; and the missing transverse energy, which stands in for the undetected neutrino. Seven high-level features are functions of these, namely invariant masses of particle combinations. The label is 1 for signal and 0 for background.

The two kinds of error have different consequences. A false positive keeps a background event that looks like signal and contaminates the candidate sample. A false negative discards a genuine signal event and reduces sensitivity. The decision threshold controls this trade-off, so one accuracy figure can hide important behaviour. For this reason the project reports ranking metrics (ROC-AUC and PR-AUC), calibration (Brier score and calibration curve) and an operating-point view, in which the threshold is chosen on validation data and background rejection is reported on test data.

The project has three motivations: to show how the choice of features affects performance; to compare a neural network with a linear baseline and a tree ensemble under one protocol; and to make results inspectable through a dashboard that cannot show numbers unless a recorded run produced them.

6. Project Objectives

The objectives follow the project charter and milestone plan. Their status is given in Section 19 and in the bug tracker.

- Load and validate the HIGGS dataset, recording its source, schema and checksum.
- Document provenance, schema and sampling in a data card.
- Perform exploratory data analysis on a documented sample.
- Implement classical baselines: Logistic Regression and XGBoost.
- Implement a compact multilayer perceptron with batch normalization and dropout.
- Compare performance across the low-level, high-level and combined feature groups.
- Evaluate ranking quality, calibration, operating points and training time.
- Establish reproducible splits, seeds and training procedures.
- Provide a dashboard that presents saved results, handles missing or invalid files and exports tables.
- Provide tests, setup instructions, a bug tracker and a changelog.

7. Scope of the Project

7.1 Within scope

Benchmark classification on documented subsets of the HIGGS dataset; Logistic Regression, XGBoost and the MLP; comparison across three feature groups; a shared split and artifact format; a Streamlit dashboard for saved results; automated tests for data schema, splits, metrics and repository hygiene; and setup and reproducibility documentation.

7.2 Outside scope

- Validation against real detector data, and any claim about real collision data.
- Reconstruction of particles or event kinematics from detector signals.
- Discovery claims, significance estimates or treatment of systematic uncertainties.
- Production hosting, user accounts or live model serving.

7.3 Planned or incomplete

Learning-curve figures for the MLP (the per-epoch history is not saved to a file); the operating-point results for the 1M benchmark; the PyTorch unit tests and the end-to-end test on a machine with PyTorch; the check of the raw HIGGS file with the smoke script; screenshots of the dashboard; and the deployment rollback test (END-06). Section 21 lists each item with its owner.

8. Dataset Description and Data Card

The dataset is the HIGGS benchmark of the UCI Machine Learning Repository [4], [5], released under the Creative Commons Attribution 4.0 licence. Table 1 separates what has been verified from what is still pending. No value has been estimated to fill a gap.

Table 1. Data card for the HIGGS dataset as used in HiggsLens

Item	Value	Status
Dataset name and source	HIGGS, UCI Machine Learning Repository, dataset id 280	Verified (UCI page)
Creator	Daniel Whiteson (2014)	Verified (UCI page)
Licence	CC BY 4.0	Verified (UCI page)
Raw file	HIGGS.csv.gz; 2,816,407,858 bytes	Recorded in 250k manifest
SHA-256 of raw file	e3c49164aba986e5c46abd64f7d0d7de8304f7c1fbc97f3197e3aaf272392649	From manifest; re-verify on the final file
Retrieval date	**[PENDING: retrieval date]**	Not recorded
Documented row count	About 11,000,000	Documented
Exact row count of raw file	**[PENDING: exact count, Aditya]**	Not confirmed
Columns	29: one label and 28 features	Confirmed on the 250k subset
Header row and missing values	None expected; none reported for the 250k subset	**[PENDING: real-file smoke test output]**
Target and encoding	Column 0; 1 = signal, 0 = background	Verified (UCI description, loader)
250k subset	250,000 rows, seeded random sampling, seed 42	Recorded in manifest
1M sample	1,000,000 rows, drawn by the same extraction function	Sample file not in repository (BUG-020); seed and method to confirm (T2)
Class counts, 250k subset	Signal 131,942 (52.78%); background 118,058 (47.22%)	Measured (manifest)
Later-chunk check, 250k	50,000 rows from offset 500,000; signal 52.95%; difference 0.18 points	Measured (manifest)
b-tag values	Discrete values for jets 1 to 4	Reported in changelog; EDA outputs not saved
Feature groups	low_level 21; high_level 7; all 28	Verified (feature dictionary and code)
Split	70% train / 15% validation / 15% test; seed 42; protocol higgs-split-v1	Verified in code; 175,000 / 37,500 / 37,500 rows

Sources: data/processed manifest; docs/09_DATASET_GUIDE.md; src/higgs_event_lab/protocol.py. The manifest also records the path of the raw file on the machine that produced it. That path is intentionally not reproduced in this report (BUG-026).

8.1 Feature dictionary

The 28 features follow the UCI column order. The full dictionary is stored in docs/feature_dictionary.csv and summarised in Appendix A. Note that the 250k metrics files list the features as feat_0 to feat_27 rather than by name (BUG-019).

8.2 Preprocessing assumptions

Transverse momenta and invariant masses are expected to be positive and right-skewed. Pseudorapidity is expected to be roughly symmetric about zero, and azimuthal angles are periodic. These are expectations from physics. The EDA is the place to confirm them, and its outputs are still pending (Section 9).

9. Exploratory Data Analysis

The exploratory analysis is in notebooks/01_midsem/01_eda_and_feature_groups.ipynb. Its aims were to confirm the schema, measure class balance and b-tag values, inspect distributions and correlations, and check whether the first rows of the file are representative.

The notebook contains ten code cells and the changelog describes its findings. However, **the notebook has no saved cell outputs**, so its summary statistics, histograms and correlation results cannot be checked from the repository. They are listed below as pending. The class-balance figures in Section 9.1 come from the data manifest, not from the notebook.

9.1 Class balance (measured)

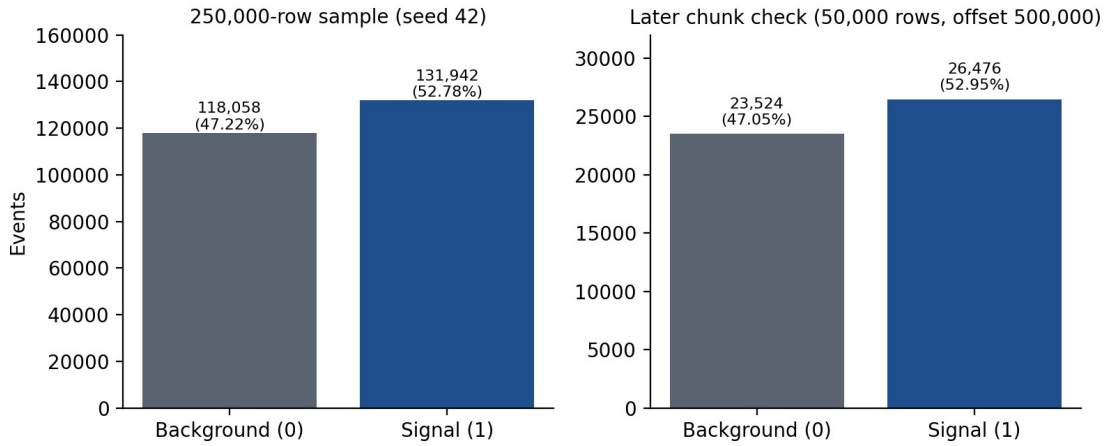


Figure 2. Measured class counts in the 250,000-row seeded sample (left) and a 50,000-row later chunk (right). Source: data manifest.

The two proportions differ by 0.18 percentage points. This supports the view that the seeded sample is not strongly biased toward either class. It is a single comparison, not proof of representativeness.

9.2 Pending EDA results

EDA item	Status
Summary statistics for the 28 features	**[PENDING: EDA notebook outputs]**
Feature distributions (histograms)	**[PENDING: EDA notebook outputs]**
Correlation analysis	**[PENDING: EDA notebook outputs]**
Exact distinct b-tag values	**[PENDING: EDA notebook outputs]**
Outlier and extreme-value analysis	**[PENDING: EDA notebook outputs]**
Feature-group comparison plots	**[PENDING: EDA notebook outputs]**

10. Proposed and Implemented Methodology

The study was run under two protocols. Protocol A is the 250,000-row benchmark, produced by the package code in src/higgs_event_lab (classical.py and neural.py) with the default or lightly set configurations. Protocol B is the 1,000,000-row benchmark, produced by scripts/train_full_colab.py with wider or tuned configurations. Both use the same split protocol (higgs-split-v1, 70/15/15, seed 42) and fit the scaler on the training partition only. Figure 3 shows the common workflow.

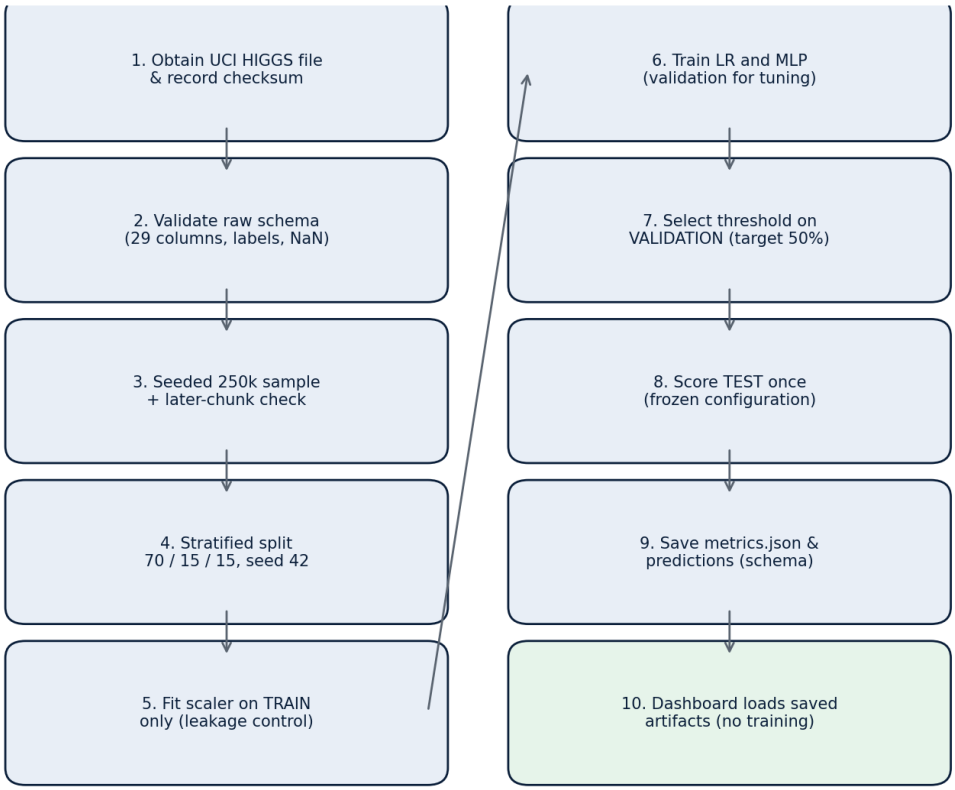


Figure 3. Methodology flowchart. Blue steps are data and training steps; green steps are dashboard steps.

10.1 Stages

- Obtain the UCI file and record its checksum and retrieval date (pending in part).
- Validate the raw schema: 29 columns, labels in {0, 1}, no empty cells, no non-finite values.
- Draw a seeded sample and record the seed and a later-chunk check.
- Create the stratified split (70/15/15, seed 42) and share it across all models in the same protocol.
- Fit the scaler on training rows only.
- Train the models. Validation data guide early stopping and any tuning.
- Select the operating threshold on validation data for a 50% signal-efficiency target, then freeze it.
- Score the test partition once, with the frozen configuration.
- Write metrics.json and test_predictions.csv in the higgs-experiment-v1 format.

- Load the saved artifacts in the dashboard. The dashboard does not train models.

10.2 Fair comparison and its limits

Within one protocol, every model uses the same split indices, so the nine feature-model combinations of Protocol A are compared on identical test events. Protocol B is not directly comparable with Protocol A: its models use different configurations (Section 13 and 14), and its results come from a different code path. Comparisons across the two protocols are therefore reported as observations, not as rankings.

10.3 Leakage controls

The controls in the code are: the scaler is fitted on training rows only in both protocols; early stopping uses validation ROC-AUC only; the MLP restores its best validation weights before test scoring; and the label is kept out of the feature matrix. The reviewed tests check the label separation. No automated test yet checks for duplicate rows across the three partitions, which is one of the hypotheses in Section 20.4.

11. System and Project Architecture

The system has four layers that communicate through files rather than shared memory (Figure 1). The main design decision is that the presentation layer reads artifacts and never retrains a model. The dashboard therefore starts quickly, behaves the same on every machine, and cannot show a number that a recorded run did not produce.

- Data layer: loader, validator, sampler, feature groups and split (data.py, features.py, protocol.py, splits.py).
- Model layer: Logistic Regression and XGBoost (classical.py) and the MLP (neural.py). The 1M benchmark uses scripts/train_full_colab.py.
- Artifact layer: metrics.json, test_predictions.csv, curves/*.json (ROC, PR and calibration coordinates), and experiments.json (run summary including operating points).
- Presentation layer: the HiggsLens Streamlit application (app/), with a CSV export.

A cross-cutting quality layer contains the unit and integration tests, a repository-hygiene test, a notebook-style test, a Dockerfile and deployment runbook, and the documentation set.

12. Data Preprocessing and Feature Engineering

12.1 Loading

The loader reads the gzip file in chunks so the whole file is never held in memory. It checks column count, numeric values and label values, and it reports failures with the row and column at fault. Unit tests use small synthetic files that exercise the same code paths.

12.2 Feature groups

Three groups are used: low_level (21 columns, file indices 1 to 21), high_level (7 columns, indices 22 to 28) and all (28 columns). Each run records which group it used. The 1M file does not record its group (Section 20.2).

12.3 Scaling

StandardScaler is applied for Logistic Regression and the MLP in both protocols, with the scaler fitted on training rows only. XGBoost is insensitive to monotone rescaling, so scaling does not affect its splits. Heavy-tailed features were not log-transformed, and periodic angles were not encoded in the reviewed code. Both are listed as limitations.

12.4 Engineered features

No features beyond the 28 supplied columns were created. The high-level invariant masses are already part of the dataset and are the closest thing to physics-based feature engineering in this project.

13. Classical Machine Learning Models

Logistic Regression is the linear baseline. It is fast and interpretable, and it shows how much a linear model can separate the classes. XGBoost is a gradient-boosted tree ensemble [8] and is the strongest classical model in this study. Both are implemented in src/higgs_event_lab/classical.py, and the 1M benchmark uses the same models with the settings in scripts/train_full_colab.py.

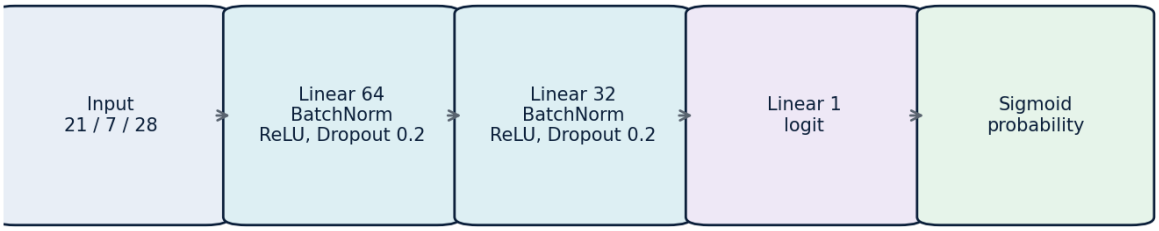
Table 2. Classical model settings by protocol (from the code)

Setting	Protocol A (250k)	Protocol B (1M)
Logistic Regression solver and C	lbfgs; C = 1.0	lbfgs (default); C = 1.0
Logistic Regression max iterations	1000	2000
XGBoost trees	150	600
XGBoost maximum depth	6	7
XGBoost learning rate	0.1	0.06
XGBoost subsample / colsample	0.8 / 0.8	**[PENDING: confirm in script]**
XGBoost tree method	hist	**[PENDING: confirm in script]**
Operating threshold	Validation-selected for 50% signal efficiency, then frozen	Not recorded in the 1M file
Training time	Recorded per run (Table 6)	**[PENDING: not reliable; see BUG-018]**

Protocol B's Logistic Regression is labelled "tuned" in its results file, but it uses the same regularisation as Protocol A and only a higher iteration limit. No tuning search is recorded for it. Protocol B's XGBoost was given more trees and greater depth than Protocol A's. These differences matter when the two protocols are compared (Section 20.3).

14. Neural Network Model

The neural model is a multilayer perceptron implemented in PyTorch (neural.py). Each hidden block is a linear layer, batch normalization, a ReLU activation and dropout. The output is a single logit, and training uses binary cross-entropy with logits.



Loss: BCEWithLogits. Optimizer: AdamW (lr 1e-3, weight decay 1e-4). Batch 1024. Early stop on validation ROC-AUC, patience 5.

Figure 4. MLP used in Protocol A (hidden widths 64 and 32, dropout 0.2). Protocol B uses a wider network (256, 128, 64; dropout 0.3).

Table 3. MLP configuration by protocol (from the code)

Setting	Protocol A (250k)	Protocol B (1M)
Input width	21, 7 or 28 by feature group	28 (all features)
Hidden widths	64, 32	256, 128, 64
Dropout	0.20	0.30
Optimiser and learning rate	AdamW, 1e-3	AdamW, 8e-4
Weight decay	1e-4	**[PENDING: confirm in script]**
Batch size	1024	**[PENDING: confirm in script]**
Maximum epochs; patience	30; 5	60; early stopping (confirm patience)
Early-stopping criterion	Validation ROC-AUC; best weights restored	Confirm in script
Seed; device	42; CPU	**[PENDING: confirm device for 1M run]**
Epochs completed	Low 30; high 10; all 20	**[PENDING: not recorded]**

14.1 Training procedure and evaluation

Each epoch trains on the training partition in shuffled mini-batches. After each epoch the model is scored on the validation partition. Training stops when validation ROC-AUC has not improved for the patience period. The best validation state is restored, and only then are validation and test probabilities computed. The test partition takes no part in any training or stopping decision.

14.2 Runtime

Protocol A runtimes were 15.02 s (low-level, 30 epochs), 5.13 s (high-level, 10 epochs) and 10.52 s (all features, 20 epochs), all on CPU. These come from the saved run metadata. Protocol B's timings are not reliable (BUG-018), so they are not reported.

14.3 Learning curves

The milestone plan asks for learning curves. The per-epoch training and validation loss are **not saved in the repository**. The curves/ files hold ROC, PR and calibration coordinates only, so learning-curve plots cannot be produced from the repository. **[PENDING: per-epoch history file from the MLP training runs (Rachit)]**

15. Model Evaluation and Performance Metrics

No single number describes a classifier. The metrics below cover ranking, precision, calibration, operating points and cost. Let TP, TN, FP and FN be the counts of true and false positives and negatives, and N the number of events.

Table 4. Evaluation metrics and their meaning

Metric	Definition	What it tells us
ROC-AUC	Probability that a random signal event scores above a random background event	Ranking quality across thresholds; 0.5 is chance
PR-AUC (average precision)	Area under the precision-recall curve	Precision for the signal class across recall; informative when the positive class is rare
Precision	TP / (TP + FP)	Purity of selected candidates
Recall (signal efficiency)	TP / (TP + FN)	Fraction of true signal kept
Confusion matrix	TP, FP, TN and FN counts at a threshold	Where the errors fall
Brier score	(1/N) sum of (p_i - y_i)^2	Mean squared error of the probabilities; lower is better
Calibration curve	Observed signal fraction within bins of predicted score	Whether scores behave like probabilities
Background rejection	TN / (TN + FP) at the frozen threshold	Background removed at the chosen operating point
Training time	Wall-clock seconds to fit	Compute cost

Accuracy is reported as a secondary figure. Because the classes are close to balanced in this benchmark, accuracy is less misleading here than it would be for a rare-event task, but it still hides the trade-off between false positives and false negatives. ROC-AUC summarises ranking across thresholds, and PR-AUC focuses on the signal class. The Brier score and the calibration curve ask whether a score of 0.8 corresponds to about 80% signal among similar events. Training time is included because a linear model and a neural network differ greatly in cost.

The operating-point analysis chooses the threshold on validation data for a 50% signal-efficiency target and reports background rejection on test data. It is a comparison measure within this benchmark. It is not a statistical significance, and it must not be read as one.

16. Software Architecture and Dashboard

HiggsLens is a Streamlit application in app/. It was built by Ronit Singh. The README describes six views: Overview, Physics Explorer, Dataset Observatory, Experiment Comparison, Validation (ROC and precision-recall curves, calibration, and a threshold workbench) and Methods and Research. The app folder also contains a seventh view module, simulation.py, whose place in the navigation still needs to be confirmed (BUG-021).

16.1 Behaviour

- Startup reads saved files and does not train models.
- Empty state: when no artifacts exist, the app shows how to produce them, not placeholder scores.
- Input validation: the loader checks the artifact format and reports incompatible files. Unit tests cover this path (26 dashboard tests passed in the review).
- Export: a CSV export of the comparison table is implemented in the results view.

16.2 Startup command

From the repository root, with the environment activated: streamlit run app/main.py. The root file streamlit_app.py is the entry point used for Streamlit Community Cloud. Both load the same code.

16.3 Screenshots

[PENDING: screenshots of HiggsLens running with the saved artifacts: overview, empty state, experiment comparison, validation view (Ronit Singh)]

16.4 Cleanup items

The repository holds two loader implementations (app/loader.py and app/utils/loader.py) and two entry points (app/main.py and streamlit_app.py). They should be consolidated before submission (BUG-021).

17. Work Division and Team Contributions

Table 5 describes the intended division of responsibilities. An item is listed as complete only where repository or team evidence confirms it.

Table 5. Work division

Team Member	Enrollment No.	Role	Responsibilities	Deliverables	Dependencies
Saumya Mishra	2401020057	Documentation, Testing and Reporting	Scientific introduction, dataset documentation, methodology, report writing, bug tracking, test review, setup guide and results documentation	Project report, technical documentation, test records and bug tracker	Verified outputs from Aditya and Rachit; decisions from Krishna
Aditya Rana	2401010034	Data and Classical ML	Dataset validation, preprocessing, sampling, feature groups, EDA, Logistic Regression and XGBoost	Data card, validation logs, EDA results, classical metrics and commands	Agreed split, seed and artifact format
Rachit Gupta	2401010363	Deep Learning	MLP architecture, training, validation, calibration, learning curves and reproducibility	Model code, configuration, metrics, loss history and prediction artifacts	Shared loader, feature groups, split and artifact format
Ronit Singh	2401010393	Dashboard and Visualization	Streamlit interface, saved-artifact loading, charts, empty states, export and malformed-file handling	Working dashboard, screenshots, export specification and startup instructions	Agreed artifact format and saved outputs
Krishna	2401010237	Integration and Technical Decisions	Approving shared specifications, resolving project decisions, integration, merge review and end-to-end validation	Approved architecture and artifact contract, integration report, final test results	Contributions from all members

17.1 Dependencies

The loader and feature groups are shared by Aditya's and Rachit's models. The split and seed underpin every comparison. The higgs-experiment-v1 artifact format is the contract between the models and the dashboard. The unresolved headline question (Section 20.4) depends on Aditya's and Rachit's reruns and on Krishna's decision, so the final results section cannot be closed until those are complete.

[PENDING: each member's confirmation of their own description in this section]

18. Implementation and Reproducibility

The commands below use the flags defined in the code. Commands that are not confirmed by the reviewed material are marked for confirmation.

18.1 Environment

Use Python 3.10 or later. Create a virtual environment and run pip install -r requirements.txt. Streamlit is needed for the dashboard and for one unit test, and PyTorch is needed for the neural tests.

18.2 Commands

Table 6. Execution commands (Protocol A unless marked)

Step	Command	Output
Check raw file (first rows)	python tests/integration/smoke_check_raw_file.py data/raw/HIGGS.csv.gz --rows 100000	PASS/FAIL report (not yet run on the real file)
Create the 250k subset	PYTHONPATH=src python -m higgs_event_lab.data --source data/raw/HIGGS.csv.gz --output data/processed/higgs_sample_250k.csv.gz --rows 250000 --seed 42	Subset and manifest
Logistic Regression and XGBoost, all groups	PYTHONPATH=src python -m higgs_event_lab.classical --data data/processed/higgs_sample_250k.csv.gz --rows 250000 --all-groups --output-dir outputs	outputs/logreg_*/ and outputs/xgboost_*/ (metrics.json, test_predictions.csv)
MLP, all groups	PYTHONPATH=src python -m higgs_event_lab.neural --data data/processed/higgs_sample_250k.csv.gz --rows 250000 --all-groups --output-dir outputs --seed 42 --epochs 30	outputs/mlp_*/ (metrics.json, test_predictions.csv)
1M benchmark (Protocol B)	Run notebooks/03_full/00_full_higgs_colab.ipynb, which calls scripts/train_full_colab.py with HIGGS_SAMPLE_PATH set	outputs/full_colab_results.json and model weights
Dashboard	streamlit run app/main.py	Local web application
Unit tests	python -m pytest tests/unit -q	Pass/fail summary
Container build	docker build -t higgs-event-lab:latest -f deployment/Dockerfile .	Container image

Flags were taken from the argparse definitions. The default dataset URL in scripts/download_data.py should be confirmed before the final run.

18.3 Artifact locations

Results are written to outputs/. The directory is excluded from version control, except for the small files listed in outputs/README.md. The raw and processed sample files and the model weights are not in the repository (BUG-020), so the results cannot be regenerated from the repository alone.

18.4 Artifact format

The format higgs-experiment-v1 is defined in src/higgs_event_lab/artifacts.py and described in docs/13_EXPERIMENT_PROTOCOL_AND_ARTIFACTS.md. Its top-level fields in each metrics.json are schema_version, run_id, created_at, model, features, data, split, training and evaluation. The approval of the format is still a draft (BUG-024).

19. Testing and Validation

Table 7 records what was run in the review environment and its status. A test that was not run is never marked as passed.

Table 7. Test results (review environment: Python 3.12, PyTorch and the real HIGGS file not available)

ID	Component	Test file	Tests	Status
T01	Feature dictionary and column order	test_feature_dictionary.py	8	PASS
T02	Repository hygiene	test_repo_hygiene.py	11	PASS
T03	Notebook readability rules	test_notebook_style.py	7	PASS
T04	Split protocol	test_protocol.py	2	PASS
T05	Feature groups	test_features.py	6	PASS
T06	Data loader and validation	test_data_loader.py	18	PASS
T07	Evaluation metrics	test_evaluation.py	4	PASS
T08	Download and preparation	test_download.py	4	PASS
T09	Logistic Regression and XGBoost	test_classical.py	7	PASS
T10	Dashboard loader, metrics and views	test_app_loader.py, test_app_metrics.py, test_dashboard_loader.py, test_app_views.py	26 + 1	PASS for 26; 1 FAIL (Streamlit not installed in review environment)
T11	Data pipeline integration (synthetic data)	test_data_pipeline.py	4	PASS
T12	MLP unit tests	test_neural.py	3	NOT RUN (PyTorch missing)
T13	End-to-end pipeline	test_end_to_end_pipeline.py	2	NOT RUN (PyTorch missing)
T14	Raw HIGGS file check	smoke_check_raw_file.py	-	NOT RUN (file not available)
T15	Leakage (code inspection)	classical.py, neural.py, train_full_colab.py	-	PASS (inspection); no automated duplicate-row check
T16	Dashboard empty state and malformed files, in a running app	app/	-	PENDING VERIFICATION
T17	CSV export, in a running app	results view	-	PENDING VERIFICATION
T18	Reproducibility: rerun matches saved metrics	-	-	NOT RUN
T19	Consistency of 250k and 1M results	outputs/	-	FAIL: inconsistent (BUG-015)

In total, 98 tests passed in the review environment, 1 failed for the environment reason stated above, and 5 PyTorch tests and the real-file check were not run. The results consistency check (T19) is not a unit test, but it is listed because it changes what the report can claim.

19.1 What the passing tests establish

The passing tests show that the schema, feature groups, splits, loader, metrics helpers and dashboard loaders behave as specified on small inputs. They do not show that the reported model scores are correct,

since those depend on full runs. They also do not show that the notebooks execute, since their outputs are not saved.

20. Results and Comparative Analysis

Status of this section: the values below are taken from the saved files in outputs/ and were read in the review. They have not been regenerated for this report. The two protocols disagree, and no headline result is declared (Section 20.4).

20.1 Protocol A: 250,000-row subset (saved run metrics)

Table 8. Test-partition results, Protocol A, seed 42 (outputs/experiments.json)

Model	Features	ROC-AUC	PR-AUC	Brier	Accuracy	Train time (s)
Logistic Regression	low_level (21)	0.7476	0.7622	0.2035	0.681	0.07
Logistic Regression	high_level (7)	0.7661	0.7427	0.1945	0.698	0.05
Logistic Regression	all (28)	0.8369	0.8377	0.1639	0.755	0.10
XGBoost	low_level (21)	0.8152	0.8229	0.1714	0.749	12.30
XGBoost	high_level (7)	0.8379	0.8243	0.1588	0.776	7.85
XGBoost	all (28)	0.8924	0.8897	0.1252	0.818	18.40
MLP (compact)	low_level (21)	0.8000	0.8097	0.1824	0.728	15.02
MLP (compact)	high_level (7)	0.9043	0.8888	0.1184	0.824	5.13
MLP (compact)	all (28)	0.9406	0.9372	0.0945	0.865	10.52

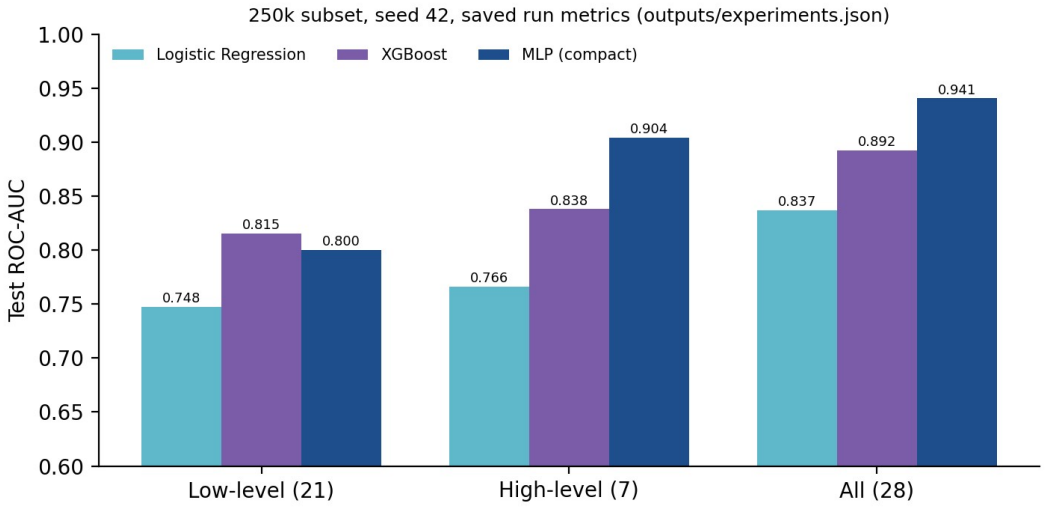


Figure 5. Test ROC-AUC by model and feature group, Protocol A. Values from saved run metrics.

20.2 Protocol B: 1,000,000-row benchmark

Table 9. Test-partition results, Protocol B (outputs/full_colab_results.json)

Model	Configuration	ROC-AUC	PR-AUC	Brier	Accuracy	Train time
Logistic Regression	C = 1.0; max iterations 2000	0.6834	0.6832	0.2233	0.641	[PENDING: BUG-018]
XGBoost	600 trees; depth 7; learning rate 0.06	0.8271	0.8415	0.1693	0.745	[PENDING:

Model	Configuration	ROC-AUC	PR-AUC	Brier	Accuracy	Train time
						BUG-018]
Wide MLP	256-128-64; dropout 0.3; up to 60 epochs	0.8399	0.8539	0.1630	0.756	[PENDING: BUG-018]

The 1M file does not record the feature group used. The results are presumed to use all 28 features, which must be confirmed (test T6 in docs/14).

20.3 Comparing the protocols

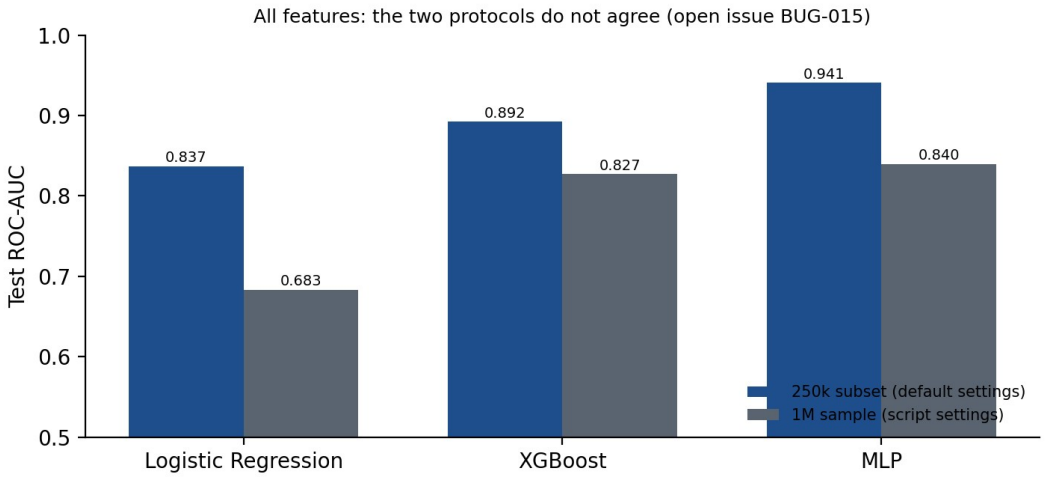


Figure 6. Test ROC-AUC on all 28 features under both protocols. The two protocols disagree (BUG-015).

Every model scores lower on Protocol B than on Protocol A. Logistic Regression falls by 0.15, XGBoost by 0.07 and the MLP by 0.10 in ROC-AUC. The Protocol B Logistic Regression uses the same regularisation as Protocol A, so tuning cannot explain its drop. The explanation must lie in the data, the feature mapping or the code path. A consistent fall across three different model families points to a difference in the pipeline rather than to any one model.

The two protocols also differ in their configurations. The Protocol B neural network is wider and trains for longer, and its XGBoost has more trees and depth. These changes would normally raise scores, so they cannot account for the fall. They do make the two protocols incomparable as rankings, however.

The Protocol A results are also high for this benchmark. The MLP's 0.9406 ROC-AUC and 0.865 accuracy are above what we recall from the original study [4] for this dataset. That is not proof of a fault, but it is the pattern that leakage would produce, and leakage would also inflate the other models on Protocol A. This is one of the hypotheses in Section 20.4.

20.4 Headline result: undecided

The report cannot yet name a headline result. The decision depends on the tests listed in **docs/14_RESULTS_RECONCILIATION.md**, which are not yet run. The main ones are a duplicate-row and partition-overlap check on the Protocol A split (T1); a clean rerun of the all-features MLP with seed 42 (T4); and running the Protocol B pipeline on the Protocol A subset to separate the data from the code (T5). Until

those are complete and Krishna records the decision, results should be quoted with their protocol, and no ranking across protocols should be stated.

Reconciliation item	Status	Owner
T1: duplicate rows and partition overlap (Protocol A)	[PENDING: not run]	Aditya
T2: 1M sample method, seed and file contents	[PENDING: not run]	Aditya
T3: code path comparison (classical.py vs train_full_colab.py)	[PENDING: not run]	Aditya
T4: clean rerun of mlp_all, seed 42	[PENDING: not run]	Rachit
T5: Protocol B code on the Protocol A subset	[PENDING: not run]	Rachit with Krishna
T6: feature group and tuning budget for each Protocol B run	[PENDING: not recorded]	Aditya, Rachit
T7: accuracy versus ROC-AUC on saved predictions	[PENDING: not run]	Rachit
Decision on the headline protocol	[PENDING: Krishna]	Krishna

20.5 Operating point (Protocol A, 50% signal-efficiency target)

The threshold was chosen on validation data for a 50% signal-efficiency target, then frozen and applied to the test partition. Table 10 gives the saved values. Background rejection is the fraction of background events removed at that threshold.

Table 10. Operating point on the test partition, Protocol A (outputs/experiments.json)

Model	Features	Threshold	Signal efficiency	Background rejection
Logistic Regression	low_level	0.510	0.501	0.795
Logistic Regression	high_level	0.519	0.497	0.812
Logistic Regression	all	0.528	0.502	0.865
XGBoost	low_level	0.521	0.502	0.858
XGBoost	high_level	0.531	0.503	0.897
XGBoost	all	0.558	0.501	0.941
MLP (compact)	low_level	0.512	0.504	0.841
MLP (compact)	high_level	0.541	0.498	0.925
MLP (compact)	all	0.582	0.501	0.962

Within Protocol A, the operating points show the same ordering as ROC-AUC: the all-features MLP removes the most background at 50% signal efficiency. This is a benchmark observation, not a significance estimate. Protocol B's operating points are not recorded (Section 7.3).

20.6 Scientific interpretation

Even on the verified Protocol A figures, the comparison shows which feature representation and model separate the simulated classes best under that protocol. It does not show that any of these models would separate real collision data, and it says nothing about the existence or properties of any particle.

21. Limitations, Risks and Open Issues

The table lists the open items in the bug tracker that affect this report, with their classification. Critical items block the final results. Cleanup items do not change any result.

Table 11. Open issues affecting the report

Issue	Impact	Resolution	Class
Protocols disagree on the headline (BUG-015)	No headline result can be claimed	Tests T1 to T5; Krishna's decision	Critical
Logistic Regression drop from 0.84 to 0.68 unexplained (BUG-016)	Data or code path may differ	Tests T2, T3, T5	Critical
High MLP accuracy with ROC-AUC 0.94 (BUG-017)	Possible leakage	T1, T4, T7	Critical
Raw-file smoke check and PyTorch tests not run	Raw-data and neural claims provisional	Run on the team machine; paste output in docs/12	Critical
Sample files and model weights not in repository (BUG-020)	Results cannot be regenerated from the repo	Document regeneration steps; commit manifests and hashes	Required
EDA and neural notebooks have no saved outputs	EDA and training claims cannot be checked	Execute and save outputs	Required
Protocol B timings copied (BUG-018)	No per-model training time for 1M	Fix the reporting code and rerun	Required
Learning curves not saved (Section 14.3)	Learning-curve figure missing	Save per-epoch history	Required
Operating point not recorded for Protocol B	No 1M rejection figures	Save with the run	Required
Dashboard screenshots missing (Section 16.3)	Presentation incomplete	Capture screenshots	Required
Feature names generic in 250k metrics (BUG-019)	Dashboard shows feat_0 to feat_27	Store the real names	Cleanup
Two dashboard entry points and two loaders (BUG-021)	Maintenance risk	Consolidate	Cleanup
Artifact format still a draft (BUG-024)	Contract not formally agreed	Krishna approves or revises	Required
Raw-file path in manifest (BUG-026)	Personal path in the repository	Strip the path	Cleanup
Repository owner placeholder (BUG-025)	Deployment may use wrong account	Confirm owner	Required before deployment

21.1 Scientific limitations

- The benchmark is simulated and describes a hypothetical signal. Results do not transfer directly to real detector data.
- The signal fraction is about 53%, whereas real searches involve far rarer signal. Precision and calibration here do not carry over to real event rates.
- Classifier scores are not physical probabilities.
- Background rejection is not a significance measure, and no systematic uncertainties are modelled.

21.2 Methodological limitations

- The two protocols use different configurations and code paths, so they are not directly comparable (Section 10.2).
- Tuning was limited. Protocol A used default or lightly set models, so a weak result may reflect under-tuning.
- Feature-group differences are confounded with model capacity and scaling.
- High-level features are functions of the low-level ones. A model that reaches high-level performance from low-level inputs shows it can learn those functions, not that the physics features are unnecessary.
- Heavy-tailed features were not transformed, and periodic angles were not encoded.
- Evaluation used one split and a small number of seeds. Uncertainty was not quantified.

22. Ethical Considerations and Scientific Interpretation

The HIGGS dataset holds simulated physics events and no personal information, so the ethical concerns are about scientific communication and honest reporting.

- Overclaiming: presenting benchmark scores as evidence of particle discovery would mislead readers. The report does not make that claim, and it repeats the limitation wherever results appear.
- Selective reporting: every configuration that was run is reported, including the linear baseline and the weaker protocol results.
- Reproducibility as honesty: numbers come from saved runs. Tests and the review checklist are designed to catch numbers typed by hand.
- Attribution: the dataset is licensed CC BY 4.0 and must be cited, together with the original study. The software licence is separate.

23. Future Scope

The following extensions are proposed and are not implemented in the repository.

- Resolve the protocol discrepancy, then run one protocol across all sample sizes that the team chooses to report.
- Hyperparameter search with an equal budget for each model, reported in full.
- Calibration analysis beyond the Brier score, including reliability diagrams for every run.
- Confidence intervals from bootstrap resampling and repeated seeds.
- Explainability methods such as SHAP, to show which features drive predictions.
- Evaluation on the dataset's documented held-out portion, for direct comparison with the published study.
- Containerised, reproducible execution with a tested rollback procedure (END-06).
- Richer dashboard filtering and comparison of runs across seeds.

24. Conclusion

HiggsLens classifies simulated Higgs-benchmark events with a reproducible pipeline. It validates the data at the level of structure and sampling, uses a shared split and an artifact format, trains Logistic Regression, XGBoost and a compact neural network across three feature groups, and presents saved results in a dashboard that does not display numbers without a run behind them.

The engineering base is sound: the code is organised, the tests that could be run passed, the leakage controls are in the code, and the documentation separates what is implemented from what is proposed. The scientific picture is not yet complete. The two protocols disagree, every model scores lower on the larger benchmark, the highest result needs an independent check for leakage, and several artifacts are missing from the repository.

Before the results can be called validated, the following must be completed: run tests T1 to T5 and record the outputs; save the learning curves, the Protocol B operating points and the training times; run the raw-file check and the PyTorch tests; capture the dashboard screenshots; approve the artifact format; and record Krishna's decision on the headline. With these in place, the work can be presented as a benchmark study of simulated events. It is not a physics discovery.

25. References

References are numbered in order of first citation. Entries marked † were not re-opened against the publisher or official page while this report was prepared. They must be checked before submission. Entries [4] and [5] (the benchmark study and the dataset page) were checked against search results during the project.

[1] CERN, “The Higgs boson,” CERN, Geneva. [Online]. Available: https://home.cern/science/physics/higgs-boson/ †

[2] ATLAS Collaboration, “Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC,” Phys. Lett. B, vol. 716, pp. 1–29, 2012, doi: 10.1016/j.physletb.2012.08.020. †

[3] CMS Collaboration, “Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC,” Phys. Lett. B, vol. 716, pp. 30–61, 2012, doi: 10.1016/j.physletb.2012.08.021. †

[4] P. Baldi, P. Sadowski, and D. Whiteson, “Searching for exotic particles in high-energy physics with deep learning,” Nature Communications, vol. 5, art. 4308, 2014, doi: 10.1038/ncomms5308. [Online]. Available: https://www.nature.com/articles/ncomms5308

[5] D. Whiteson, “HIGGS,” UCI Machine Learning Repository, 2014, doi: 10.24432/C5V312. [Online]. Available: https://archive.ics.uci.edu/dataset/280/higgs †

[6] F. Englert and R. Brout, “Broken symmetry and the mass of gauge vector mesons,” Phys. Rev. Lett., vol. 13, p. 321, 1964. † See also P. W. Higgs, Phys. Rev. Lett., vol. 13, p. 508, 1964, and G. S. Guralnik, C. R. Hagen and T. W. B. Kibble, Phys. Rev. Lett., vol. 13, p. 585, 1964. †

[7] The Nobel Prize in Physics 2013. [Online]. Available: https://www.nobelprize.org/prizes/physics/2013/summary/ †

[8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, doi: 10.1145/2939672.2939785. †

[9] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org †

[10] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems 32, 2019. [Online]. Available: https://pytorch.org †

[11] Streamlit Inc., “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io †

[12] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proc. ICLR, 2019. †

[13] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in Proc. ICML, 2015. †

[14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” J. Mach. Learn. Res., vol. 15, pp. 1929–1958, 2014. †

[15] T. Fawcett, “An introduction to ROC analysis,” Pattern Recognit. Lett., vol. 27, no. 8, pp. 861–874, 2006. †

[16] J. Davis and M. Goadrich, “The relationship between precision-recall and ROC curves,” in Proc. ICML, 2006. †

[17] G. W. Brier, “Verification of forecasts expressed in terms of probability,” Monthly Weather Review, vol. 78, no. 1, pp. 1–3, 1950. †

[18] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in Proc. ICML, 2005. †

[19] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015. †

26. Appendices

Appendix A. Dataset schema

Group	File columns	Count	Names
Label	0	1	class_label (1 = signal, 0 = background)
Low-level	1 to 21	21	lepton_pt, lepton_eta, lepton_phi, missing_energy_magnitude, missing_energy_phi, jet_1 to jet_4 (pt, eta, phi, btag)
High-level	22 to 28	7	m_jj, m_jjj, m_lv, m_jlv, m_bb, m_wbb, m_wwbb

Appendix B. Model settings

Logistic Regression, XGBoost and MLP settings for both protocols are in Tables 2 and 3 (Sections 13 and 14).

Appendix C. Execution commands

Listed in Table 6 (Section 18).

Appendix D. Artifact format (higgs-experiment-v1)

Each metrics.json holds schema_version, run_id, created_at, model, features, data, split, training and evaluation. The evaluation block holds validation and test metrics, and the test block includes calibration coordinates. The authoritative field list is docs/13_EXPERIMENT_PROTOCOL_AND_ARTIFACTS.md.

Appendix E. Test results

See Table 7 (Section 19).

Appendix F. Figures and tables

Figures

Figure	Description	Section
1	Architecture of HiggsLens as implemented	3
2	Measured class counts (250k sample and later chunk)	9
3	Methodology flowchart	10
4	MLP architecture (Protocol A)	14
5	Test ROC-AUC by model and feature group (Protocol A)	20
6	All-features ROC-AUC under both protocols	20
Cover	Schematic particle-collision background (generated illustration, not a photograph)	Cover

Tables

Table	Description	Section
1	Data card	8
2	Classical model settings by protocol	13

Table	Description	Section
3	MLP settings by protocol	14
4	Evaluation metrics	15
5	Work division	17
6	Execution commands	18
7	Test results	19
8	Protocol A results (saved runs)	20
9	Protocol B results	20
10	Operating points, Protocol A	20
11	Open issues	21

Appendix G. Team contributions

See Table 5 (Section 17). Each member should add a one-paragraph account of their own work, the files they wrote and the tests they ran.

Appendix H. Issue status

The status of each bug is in team/BUG_TRACKER.md. The register contains 26 items, of which 14 are fixed and 12 are open at the date of this report (Section 21).